

SALESTORM

API DESIGN & CONTRACT SPECIFICATION

High-Scale E-Commerce Flash-Sale Platform

Document	Baseline
05_API	API Design v1.0
Source	SALESTORM Requirements & Assumptions v2.0
Purpose	Concrete API contracts for the purchase path

Purpose

Define how clients, SALESTORM services and external providers communicate while preserving the v2.0 guarantees for inventory, reservation, payment, order creation, recovery, security and fairness.

Important: the v2.0 requirements define required behaviour and guarantees, not fixed URI names. Endpoint names in this document are proposed contracts derived from those requirements.

1. API Scope & Design Principles

The API layer is the controlled boundary between flash-sale traffic and the correctness-critical services. It must absorb edge traffic, enforce admission/fairness, validate requests and expose deterministic business outcomes.

```
CLIENT
↓
CDN / WAF / DDoS
↓
Virtual Waiting Room / Admission Control
↓ signed short-lived sale token
API GATEWAY
■■ AuthN/AuthZ
■■ Rate limits / abuse controls
■■ Schema validation
■■ Correlation / tracing
■■ Routing
■■ Product / Sale
■■ Cart
■■ Inventory / Reservation
■■ Checkout / Payment
■■ Order / Admin
↓
Transactional Outbox → Message Broker → Workers / Consumers
```

Principle	API decision
Correctness first	Authoritative inventory store decides reservation; cache is advisory.
Idempotency	Buy, payment, refund and callbacks use stable idempotency/event keys.
Fail safe	Unknown payment state is reconciled before retry/release.
Separate outcomes	409/410 sold-out or state conflicts; 429 rate limit; 503 overload.
Async recovery	Order recovery, fulfilment, notifications and reconciliation use events.
Traceability	Correlation/trace ID spans request → reservation → payment → order → notification.

2. Common API Standards

Standard	Decision
Base path	/api/v1
Transport	HTTPS; TLS 1.2+ in transit.
Customer auth	OAuth2/OIDC + JWT.
Service auth	mTLS or signed service tokens.
Admin auth	RBAC least privilege + MFA; sensitive stock changes may require dual control.
Tracing	X-Correlation-ID / distributed trace context.
Idempotency	Idempotency-Key on critical state-changing operations.
Time	ISO-8601 UTC; trusted server clock for reservation expiry.
Caching	Catalogue/sale reads cacheable; inventory cache never authoritative.
Payment data	No PAN/card data reaches SALESTORM; hosted/tokenized payment.
Retries	Bounded jittered exponential backoff; Retry-After where appropriate.

Recommended headers

```
Authorization: Bearer <JWT>
Idempotency-Key: <unique-operation-key>
X-Correlation-ID: <trace-id>
X-Sale-Token: <signed-short-lived-admission-token>
Content-Type: application/json
```

The sale token implements the v2.0 admission-control requirement: admitted users hold a signed, short-lived token before reaching the reservation path.

3. Product, Sale & Cart APIs

Method	Endpoint	Purpose
GET	/api/v1/products/{productId}	Product + sale information.
GET	/api/v1/sales/{saleId}	Sale window, eligibility and customer-visible deal information.
GET	/api/v1/products/{productId}/availability	IN_STOCK / LOW / SOLD_OUT advisory signal.
GET	/api/v1/carts/{cartId}	Read cart.
POST	/api/v1/carts/{cartId}/items	Add item; does not reserve stock.
PATCH	/api/v1/carts/{cartId}/items/{itemId}	Update cart quantity.
DELETE	/api/v1/carts/{cartId}/items/{itemId}	Remove cart item.

Availability response

```
{
  "productId": "P1001",
  "stockStatus": "LOW",
  "price": 79999,
  "currency": "INR",
  "version": 42,
  "asOf": "2026-10-05T07:30:00Z"
}
```

Rule: inventory availability is advisory. The store of record decides whether a reservation succeeds. Cart state is never stock ownership.

4. Admission Control & Sale Entry

TRF-01 requires a virtual waiting room/admission control that caps concurrent checkout entrants. Fairness is first-admitted, not first-clicked.

Method	Endpoint	Purpose
POST	/api/v1/sales/{saleId}/admission	Join sale path; return queue position or signed admission token.
GET	/api/v1/sales/{saleId}/queue-status	Read position and approximate wait.
POST	/api/v1/sales/{saleId}/admission/refresh	Refresh a valid admission token when permitted.

Admitted response

```
{
  "status": "ADMITTED",
  "saleToken": "<signed-short-lived-token>",
  "expiresAt": "2026-10-05T07:35:00Z"
}
```

Waiting response

```
{
  "status": "WAITING",
  "queuePosition": 1832,
  "approxWaitSeconds": 74,
  "pollAfterSeconds": 5
}
```

Edge validation must verify authenticated session, sale token, bounded payload and schema before expensive origin work. Browse and Buy Now rate limits are separate.

5. Reservation / Buy Now APIs

Reservation is the primary correctness boundary. It atomically reserves stock in the authoritative inventory store and returns reservation_id and expires_at. Duplicate requests with the same idempotency key never create a second reservation.

Method	Endpoint	Purpose	Idempotency
POST	/api/v1/reservations	Create atomic reservation; evaluate/lock server-side price.	Required
GET	/api/v1/reservations/{reservationId}	Read reservation state and expiry.	—
POST	/api/v1/reservations/{reservationId}/release	Release an owned reservation; idempotent.	Required
POST	/api/v1/reservations/{reservationId}/extend	One permitted payment/3DS extension.	Required

Request

```
POST /api/v1/reservations
Idempotency-Key: BUY-P1001-<operation-id>

{"productId":"P1001","quantity":1,"couponCode":"SALE10"}
```

Success

```
201 Created
{
  "reservationId":"RSV-92831",
  "status":"RESERVED",
  "productId":"P1001",
  "quantity":1,
  "unitPrice":71999,
  "currency":"INR",
  "expiresAt":"2026-10-05T07:40:00Z",
  "version":1
}
```

Reservation lifecycle

```
AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD
■ → PAYMENT_FAILED → RELEASED
■ → TIMEOUT / EXPIRED → RELEASED

Expired + late payment → deterministic RE-ACQUIRE OR REFUND
```

6. Reservation Response & Failure Semantics

Condition	HTTP	Code	Client behaviour
Sold out / no available unit	409	SOLD_OUT	Do not retry.
Purchase limit reached	409	PURCHASE_LIMIT_REACHED	Do not retry unchanged.
Expired/invalid sale token	403/410	INVALID_SALE_TOKEN	Re-enter admission flow.
Reservation expired	410	RESERVATION_EXPIRED	Restart purchase if sale is open.
Rate limited	429	RATE_LIMITED	Honor Retry-After.
Origin/dependency overload	503	SERVICE_UNAVAILABLE	Bounded backoff.
Same key, same operation	200/201	ORIGINAL_RESULT	Return original result.
Same key, different payload	409	IDEMPOTENCY_CONFLICT	Do not execute.

The API must preserve the distinction required by TRF-06: 429/503 are retry-later conditions; 409/410 sold-out or invalid-state outcomes are not generic overload.

Authoritative transaction expectations

1. Validate identity, sale token, payload and purchase limit.
2. Read authoritative inventory record.
3. Atomically verify available quantity and create reservation.
4. Persist unique idempotency key and reservation result.
5. Append inventory ledger/audit information.
6. Commit outbox event with the state change.
7. Return reservation_id + expires_at.

The cache, counter and queue may accelerate rejection, but correctness remains in the store of record.

7. Checkout & Payment APIs

Checkout requires a valid, unexpired reservation owned by the caller. Payment uses hosted/tokenized flows; SALESTORM never receives raw card data.

Method	Endpoint	Purpose
POST	/api/v1/checkouts	Validate reservation and create checkout context.
GET	/api/v1/checkouts/{checkoutId}	Read checkout readiness.
POST	/api/v1/payments	Initiate/confirm payment using tokenized method.
GET	/api/v1/payments/{paymentId}	Read payment state.
POST	/api/v1/payments/{paymentId}/refund	Idempotent refund/void when eligible.

Payment request/response

```
{
  "reservationId": "RSV-92831",
  "paymentMethodToken": "pm_tok_abc123"
}

→ {
  "paymentId": "PAY-7712",
  "merchantTxnRef": "SALESTORM-20261005-7712",
  "status": "INITIATED",
  "amount": 71999,
  "currency": "INR",
  "nextAction": "HOSTED_CHECKOUT"
}
```

Payment state

```
INITIATED → AUTHORISED → CAPTURED
■ → FAILED
■ → TIMED_OUT → STATUS_QUERY / RECONCILE
■ → REFUND_PENDING → REFUNDED
```

Before capture, amount/currency/order total are re-verified against the locked reservation price. Timeout never causes a blind re-charge.

8. Payment Webhooks & Reconciliation

Method	Endpoint	Purpose
POST	/api/v1/webhooks/payments	Verify signed provider event, dedupe event_id, apply valid forward transition.
GET	/api/v1/payments/{paymentId}/status	Internal status query after timeout/uncertainty.
POST	/api/v1/reconciliation/payments/run	Run settlement reconciliation.
GET	/api/v1/reconciliation/payments/{runId}	Read unmatched items and reconciliation result.

Webhook example

```
{
  "eventId": "evt_9012",
  "eventType": "PAYMENT_CAPTURED",
  "merchantTxnRef": "SALESTORM-20261005-7712",
  "paymentId": "PAY-7712",
  "reservationId": "RSV-92831",
  "amount": 71999,
  "currency": "INR",
  "occurredAt": "2026-10-05T07:34:12Z"
}
```

Handling sequence: signature verification → schema validation → event-ID dedupe → correlation → forward-only state transition → audit/outbox. Duplicate or reordered callbacks must not create duplicate payment/order effects.

Daily settlement reconciliation is required; unmatched items are surfaced to Finance. Payment refunds/voids are idempotent.

9. Order APIs

Method	Endpoint	Purpose
GET	/api/v1/orders	List caller's orders.
GET	/api/v1/orders/{orderId}	Read order/payment/fulfilment status.
POST	/api/v1/orders/{orderId}/cancel	Cancel while allowed; return stock and initiate refund.
GET	/api/v1/orders/{orderId}/history	Read append-only state history.

Order lifecycle

```
CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED
↓
OUT_FOR_DELIVERY
↓
DELIVERED

PAYMENT_PENDING → PAYMENT_FAILED → CANCELLED
CONFIRMED / PROCESSING → CANCELLED → REFUND_PENDING → REFUNDED
```

Order creation is normally an internal service action after captured payment; clients do not arbitrarily create orders. Unique payment reference / reservation identity prevents duplicates.

Recovery guarantee

If payment is captured while Order Service is down, payment remains CAPTURED and the durable event remains available for retry/reconciliation. The system must create the valid order after recovery and avoid duplicates. The requirements target conversion of all paid-without-order payments within 60 seconds after recovery.

10. Fulfilment, Shipment & Notification APIs

Method	Endpoint	Purpose
POST	/api/v1/shipments	Internal asynchronous fulfilment request for confirmed order.
GET	/api/v1/shipments/{shipmentId}	Shipment/tracking state.
POST	/api/v1/webhooks/shipments	Receive partner tracking events; idempotent and ordered per shipment.
GET	/api/v1/orders/{orderId}/tracking	Customer tracking view.
POST	/api/v1/notifications	Internal notification command/event consumer.
Event		Customer-visible milestone
RESERVATION_CREATED		Reservation and expiry
PAYMENT_PENDING		Payment in progress
PAYMENT_CAPTURED		Payment received
ORDER_CONFIRMED		Order confirmed
ORDER_PROCESSING		Fulfilment started
SHIPPED / OUT_FOR_DELIVERY / DELIVERED		Shipment lifecycle
REFUND_ISSUED		Refund status

Delivery/notification failure must not roll back a paid order. Retry, fallback partner/channel or manual escalation is used.

11. Admin & Sale Control APIs

Method	Endpoint	Purpose
POST	/api/v1/admin/sales	Configure sale window, allocation, price, limits, TTL and eligibility.
PATCH	/api/v1/admin/sales/{saleId}	Audited sale configuration change.
POST	/api/v1/admin/sales/{saleId}/pause	Pause sale.
POST	/api/v1/admin/sales/{saleId}/disable-buy	Disable Buy Now safely.
POST	/api/v1/admin/sales/{saleId}/force-sold-out	Force customer-facing sold-out state.
PATCH	/api/v1/admin/inventory/{productId}	Audited stock adjustment.
POST	/api/v1/admin/dlq/replay	Replay investigated DLQ messages.

All control-plane actions require RBAC and MFA and create tamper-evident audit entries. Stock changes above the configured threshold use dual control.

Kill-switch behaviour

```
Operator → disable Buy Now
↓
Gateway rejects new purchase admissions / reservation attempts
↓
Existing reservations continue safely
↓
Payment/order recovery paths remain active
↓
Audit + metrics + alert confirm state
```

12. Error Model, Retry & Idempotency

```
{
  "error":{
    "code":"RESERVATION_EXPIRED",
    "message":"The reservation has expired.",
    "correlationId":"corr-92831",
    "retryable":false,
    "details":{}
  }
}
```

HTTP	Meaning	Examples	Retry?
400	Invalid request	VALIDATION_ERROR	No until fixed
401	Unauthenticated	UNAUTHENTICATED	Re-authenticate
403	Forbidden	FORBIDDEN / INVALID_SALE_TOKEN	Refresh authorization if allowed
409	Business conflict	SOLD_OUT / PURCHASE_LIMIT / IDEMPOTENCY_CONFLICT	Usually no
410	No longer valid	RESERVATION_EXPIRED / SALE_CLOSED	Restart flow if allowed
429	Rate limited	RATE_LIMITED	Yes, honor Retry-After
500	Unexpected error	INTERNAL_ERROR	Only safe idempotent operations
503	Temporary overload	SERVICE_UNAVAILABLE	Bounded backoff

Idempotency contract

1. Client generates one key for one logical operation.
2. Server binds key to caller + operation + validated payload.
3. First execution persists the business result.
4. Duplicate request returns the original result.
5. Same key with different payload returns `IDEMPOTENCY_CONFLICT`.
6. Gateway/payment references and unique database constraints backstop the API.

Retry rule: clients must never retry `SOLD_OUT` as though it were overload. This protects the hot inventory path from retry storms.

13. Asynchronous Event Contracts

Messaging uses at-least-once delivery with idempotent consumers, producing an effectively-once business effect.

Field	Purpose
event_id	Global dedupe identifier.
event_type	Versioned event name.
aggregate_id	Reservation/order/payment identity.
version	Schema/event version.
correlation_id	End-to-end trace identifier.
causation_id	Request/event that caused this event.
timestamp	UTC event time.
payload	Versioned business data; consumers tolerate unknown fields.

Event	Producer	Consumers
RESERVATION_CREATED	Inventory	Checkout / notification
RESERVATION_EXPIRED	Reservation worker	Inventory / notification
PAYMENT_CAPTURED	Payment	Order / notification
PAYMENT_FAILED	Payment	Reservation / order
PAYMENT_TIMED_OUT	Payment	Reconciliation
ORDER_CREATED / CONFIRMED	Order	Fulfilment / notification
ORDER_CANCELLED	Order	Inventory / refund / notification
REFUND_ISSUED	Payment	Order / notification / finance
SHIPMENT_UPDATED	Fulfilment	Order / tracking / notification

State change + event are committed atomically via transactional outbox/equivalent. Retries use exponential backoff with jitter; bounded failures enter a DLQ with alerting and replay. Per-order/reservation/payment ordering is preserved where required.

14. Critical API Call Flows

14.1 Buy Now → Payment → Order

```
Customer → Gateway → Admission → Reservation
■
■ → authoritative inventory transaction
■
reservation_id + expiry
↓
Customer → Checkout → Payment Service → Gateway
■
CAPTURED
↓
Outbox → Broker → Order Service
■
Order Created
↓
Fulfilment / Notification
```

14.2 Payment timeout

```
Payment Service → Gateway
■
■■ timeout
■
■■ status query by merchantTxnRef
■■ CAPTURED → continue order flow
■■ FAILED → release reservation
■■ UNKNOWN → reconcile / query again
(never blind re-charge)
```

14.3 Reservation expiry

```
Expiry Worker
→ find expires_at <= trusted_now
→ atomic RESERVED → EXPIRED/RELEASED
→ inventory ledger entry
→ outbox → RESERVATION_EXPIRED
→ released unit becomes available again
```

14.4 Order service outage

```
Payment CAPTURED
→ durable outbox event
→ Order Service unavailable
→ event remains durable
→ service recovers
→ idempotent order creation
→ paid-without-order reconciliation = zero unexplained records
```

15. Requirement-to-API Traceability

Requirement area	API evidence
FR-01...06	Discovery, cart, availability, reservation, expiry and release contracts.
FR-07...11	Reservation/payment idempotency + payment-captured → exactly-one-order path.
FR-12...18	Order state APIs, recovery, fulfilment, tracking and notification events.
FR-19...24	RBAC admin sale configuration, pricing lock, cancellation, refund, status and kill-switch APIs.
INV-01...10	Authoritative reservation transaction, purchase limits, idempotency, release and audited inventory adjustments.
PAY-01...11	Tokenized payment, unique references, signed webhooks, timeout query, 3DS extension, refund and reconciliation.
ORD-01...11	Forward-only order lifecycle, unique payment/reservation relation, outbox, cancellation, fulfilment and notifications.
MSG-01...07	At-least-once, idempotent consumers, versioned events, ordering, retries, DLQ and event metadata.
TRF-01...09	Admission APIs, signed sale token, rate limits, WAF validation, 409/410 vs 429/503, Retry-After.
SEC-01...10	JWT/RBAC, service identity, TLS, audit, vault, PII controls, replay/CSRF protection and validation.

Performance targets carried into API review: up to 500,000 req/s at edge, origin dynamic traffic $\leq 25,000$ req/s, $\geq 95\%$ edge/cache service target, reservation decision p99 ≤ 300 ms and p50 ≤ 80 ms at gateway, cached read p99 ≤ 200 ms, 5xx $< 0.1\%$.

16. API Review Checklist

Check	Pass condition
Admission	Buy Now requires a valid short-lived admission/sale token.
Inventory	No API treats cache/counter as authoritative stock.
Idempotency	Reservation/payment/refund/callback retries cannot duplicate business effects.
Price	Client price is never authoritative; reservation locks server-evaluated price.
Payment	Timeouts query/reconcile; no blind re-charge.
Webhooks	Signed, deduplicated, replay-protected, forward-only.
Order	Every captured payment has a durable path to one valid order.
Events	State change + event commit atomically; consumers idempotent.
Errors	409/410, 429 and 503 have distinct semantics.
Security	TLS, JWT/RBAC, service identity, validation and audit.
Observability	Correlation ID spans request → reservation → payment → order → notification.
Operations	Pause/disable/force-sold-out and DLQ replay are audited.
Recovery	DB, broker, gateway and Order Service failure paths are explicit and testable.

Next artifact: Overall API Call Design

Use this PDF as the contract baseline. The next visual should show the overall API call flow across Client → CDN/WAF → Admission → API Gateway → Inventory/Reservation → Payment → Order → Fulfilment, clearly separating synchronous calls, webhooks and asynchronous broker events.

END OF 05_API — API DESIGN & CONTRACT SPECIFICATION